moore_fsm_source_code

top_moore_fsm.v

```
`timescale 1ns / 1ps

module top_moore_fsm(
    input clk,
    input reset, // 비동기 reset, active-high
    input btnU,
    input btnD,
    output led
    );

    wire w_clean_btnU;
    wire w_clean_btnD;

    debouncer u_btn_debouncer_U(
        .clk(clk),
        .reset(reset),
        .noisy_btn(btnU), // raw noisy button input
        .clean_btn(w_clean_btnU)
    );

    debouncer u_btn_debouncer_D(
        .clk(clk),
        .reset(reset),
        .noisy_btn(btnD), // raw noisy button input
        .clean_btn(w_clean_btnD)
    );

    moor_fsm_101 u_moor_fsm_101(
        .clk(clk),
        .reset(reset),
        .x2(w_clean_btnD),
        .x1(w_clean_btnU),
        .y(led)
    );
```

```
endmodule
```

moore_fsm_101.v

```
`timescale 1ns / 1ps

module moor_fsm_101(
    input clk,
    input reset,
    input x1,   // Up 버튼 (디바운싱 완료)
    input x2,   // Down 버튼 (디바운싱 완료)
    output y
);

    // 상태 정의
    localparam [1:0] s0 = 2'b00,
                     s1 = 2'b01,
                     s2 = 2'b10,
                     s3 = 2'b11;

    reg [1:0] curr_state, next_state;

    // 이전 버튼 상태 저장
    reg prev_x1, prev_x2;

    always @(posedge clk or posedge reset) begin
        if (reset) begin
            prev_x1 <= 0;
            prev_x2 <= 0;
        end
        else begin
            prev_x1 <= x1;
            prev_x2 <= x2;
        end
    end

    // 1. 현재 상태 저장
```

```
always @(posedge clk or posedge reset) begin
    if (reset)
        curr_state <= s0;
    else if ((x1 && x1 != prev_x1) ||
            (x2 && x2 != prev_x2))
        curr_state <= next_state;
end

// 2. 다음 상태 결정
always @(*) begin
    next_state = curr_state;

    case (curr_state)
        s0: if (x1 && x1 != prev_x1)
                next_state = s1;

        s1: if (x2 && x2 != prev_x2)
                next_state = s2;

        s2: begin
            if (x1 && x1 != prev_x1)
                next_state = s3;
            else if (x2 && x2 != prev_x2)
                next_state = s0;
        end

        s3: begin
            if (x1 && x1 != prev_x1)
                next_state = s1;
            else if (x2 && x2 != prev_x2)
                next_state = s2;
        end

        default: next_state = s0;
    endcase
end

// 3. Moore 출력
assign y = (curr_state == s3);
```

```
endmodule
```